

RAPPORT ET CODE SOURCE

Groupe N° : 3

Membres du groupe :

- BACHAA Hajar
- BEN LTAIF Yasmine
- DEVAUD Antoine
- GRELET Thomas
- YAHYAOUI Nidal

Tuteur :

- VANDERSTRAETEN Julien

Année universitaire 2025-2026

1. Introduction et Mise en Contexte.....	3
1.1. Présentation du PFR.....	3
1.2. Objectifs et Défis.....	3
1.3. Environnement de travail.....	3
2. Architecture Logicielle Globale.....	4
2.1. Schéma des flux de données du système.....	4
2.2. Description de l'Architecture Logicielle.....	4
3. Étude Technique : Le Cœur en C.....	5
3.1. Module de Commande Vocale et Textuelle.....	5
3.2. Module de Traitement d'Image.....	5
3.3. Gestion de la Compilation.....	6
4. Étude Technique : Simulation et Visualisation (Python).....	6
4.1. Modélisation du Robot.....	6
4.2. Moteur d'Environnement : Murs, Ouvertures et Obstacles.....	7
4.3. Algorithmes de Navigation et de Recherche de Balles.....	7
5. Validation et Résultats.....	7
5.1. Scénarios de tests effectués.....	7
5.2. Analyse des résultats obtenus.....	8
5.3. Limites rencontrées.....	8
6. Gestion de Projet et Organisation.....	9
6.1. Répartition des tâches au sein du groupe.....	9
6.2. Outils de collaboration.....	9
6.3. Communication et gestion des versions.....	9
7. Bilans Individuels.....	10
7.1. Bilan de BACHAA Hajar.....	10
7.2. Bilan de BEN LTAIEF Yasmine.....	11
7.4. Bilan de GRELET Thomas.....	14
7.5. Bilan de YAHYAOUI Nidal.....	16
8. Conclusion.....	18
8.1. Bilan général du projet.....	18
8.2. Perspectives d'évolution.....	18

1. Introduction et Mise en Contexte

Le Projet Fil Rouge (PFR) s'inscrit dans une démarche d'intégration de compétences transversales allant de la programmation système en **C** au développement de scripts en Python. L'objectif est de concevoir un système complet permettant de piloter un robot mobile par la voix, tout en lui permettant d'interagir avec des objets détectés par traitement d'image.

1.1. Présentation du PFR

Le sujet traite de la création d'une interface de contrôle. Le robot doit pouvoir interagir avec son environnement et l'utilisateur selon trois axes technologiques majeurs :

- **L'interface de commande** : L'utilisateur peut piloter le robot soit par la voix, soit par la saisie de texte (clavier).
- **Le traitement d'image** : Identifier et localiser des obstacles (balles colorées) pour automatiser certaines tâches.
- **La simulation robotique** : Traduire ces instructions (vocales ou textuelles) en mouvements graphiques fluides.

1.2. Objectifs et Défis

Le défi technique majeur de ce projet réside dans la communication inter-langages. Il s'agit de faire cohabiter la performance du langage C pour le traitement des données brutes (audio/image) avec la flexibilité de Python pour la visualisation graphique. La robustesse du système repose sur sa capacité à gérer des fichiers de données partagés (**.txt**, **.json**) pour synchroniser les différents modules.

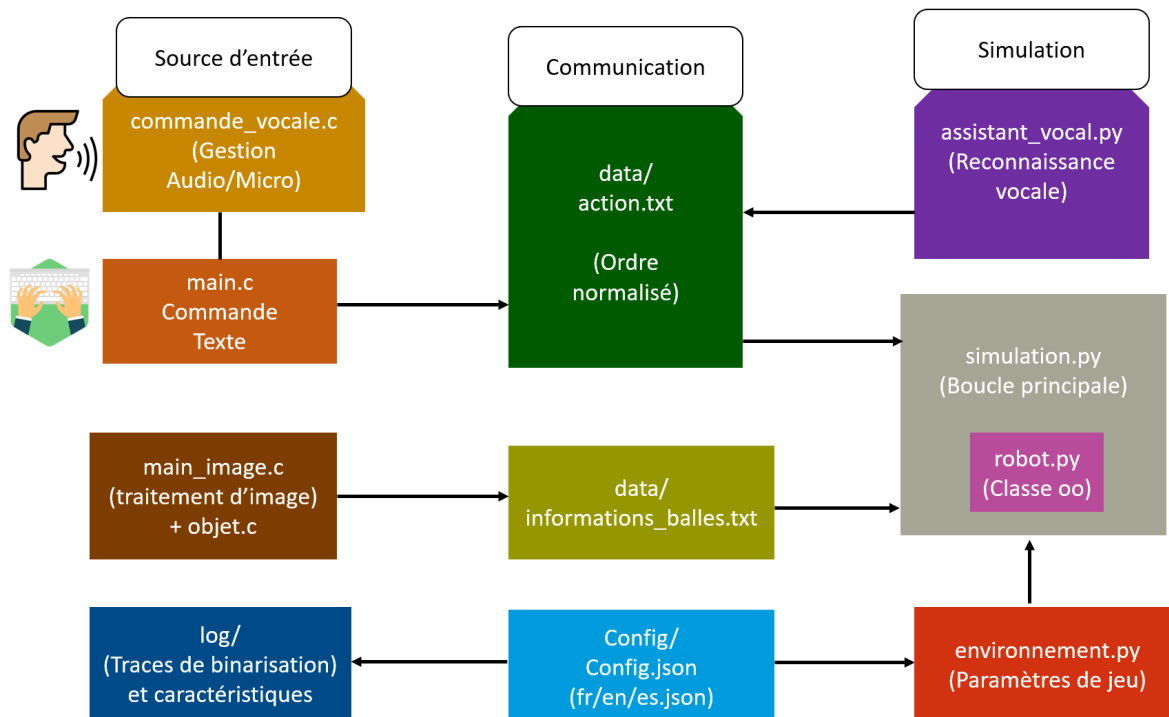
1.3. Environnement de travail

Pour mener à bien ce développement, notre groupe a utilisé les outils suivants :

- **Système** : Machine virtuelle VMware sous Linux pour la compilation et l'exécution.
- **Gestion de version** : GitHub pour le versioning et le travail collaboratif sur les fichiers de configuration JSON et les modules C/Python.

2. Architecture Logicielle Globale

2.1. Schéma des flux de données du système



2.2. Description de l'Architecture Logicielle

L'architecture du système est conçue selon une approche modulaire, divisée en trois pôles distincts : l'acquisition, la communication et la simulation. Le pôle d'acquisition regroupe les interfaces d'entrée, notamment le module de commande textuelle via `main.c` et la gestion audio/micro gérée par `commande_vocale.c`. En parallèle, le module de vision s'appuie sur `image.c` et `objet.c` pour effectuer le traitement d'image et la détection de cibles.

Le pivot de cette structure repose sur le pôle de communication, matérialisé par les fichiers tampons `data/action.txt` et `data/informations_balles.txt`. Ces fichiers assurent le transfert des ordres normalisés et des coordonnées des objets vers la couche de simulation. Cette dernière, développée en Python, est orchestrée par `simulation.py` qui exploite la classe orientée objet (oo) définie dans `robot.py` pour animer la simulation graphique.

Enfin, l'ensemble du système est piloté par des paramètres externes regroupés dans le répertoire `config/`, permettant une gestion multilingue et une configuration dynamique sans modification du code source. La traçabilité du traitement d'image est assurée par la génération de journaux d'exécution dans le répertoire `log/`, documentant chaque étape, de la binarisation à la caractérisation des formes.

3. Étude Technique : Le Cœur en C

Cette section détaille l'implémentation des modules bas niveau, où la performance du langage C est exploitée pour le traitement de données lourdes (images, flux textuels et vocaux).

3.1. Module de Commande Vocale et Textuelle

Le pilotage du robot repose sur une interaction hybride gérée par le fichier `main.c`.

- **Gestion du menu** : Le programme principal propose un menu interactif permettant de choisir entre la saisie clavier et la commande vocale.
- **Liaison Python/C** : Pour la partie vocale, le code C utilise la fonction `system()` pour appeler le script `assistant_vocal.py`, déléguant ainsi la reconnaissance sonore au module Python plus adapté à cette tâche.
- **Normalisation des ordres** : Une fois la commande reçue (texte ou voix), le module `commande_vocale.c` assure la validation de la syntaxe avant d'écrire l'ordre définitif dans le fichier tampon `data/action.txt`.

3.2. Module de Traitement d'Image

Le module de vision est le moteur de perception du robot. Il s'appuie sur les fichiers `image.c`, `objet.c` et le programme de test `main_image.c`.

- **Acquisition et analyse** : Le système lit des images en texte, qui contiennent les trois matrices Rouge, Vert et Bleu (RVB) (exemple: `IMG_5389.txt`) pour y détecter des objets colorés.
- **Chaîne de traitement** : Le processus transforme l'image brute en une série de données simplifiées. Cette progression est documentée dans le dossier `log/test_images/` via des fichiers comme `_binarisee.txt` et `_caracteristiques.txt`.
- **Localisation des cibles** : La fonction `trouver_informations_balles()` extrait la position et la couleur des objets détectés pour les consigner dans `data/informations_balles.txt`, permettant au robot de connaître son environnement.

3.3. Gestion de la Compilation

Pour automatiser la construction du projet, nous avons élaboré un **Makefile** structuré.

- **Modularité** : Chaque fichier source (.c) est compilé individuellement en fichier objet (.o) dans le répertoire **lib/**.
- **Cibles multiples** : Le Makefile propose des cibles pour générer l'exécutable principal (**bin/prog_principal.out**) ou le programme de test image (**bin/prog_image.out**).
- **Nettoyage** : Les commandes **clean** et **fclean** permettent de maintenir l'environnement de développement propre en supprimant respectivement les fichiers objets (**lib/*.o**) et les fichiers binaires (**bin/*.out**).

4. Étude Technique : Simulation et Visualisation (Python)

Cette section détaille la couche logicielle responsable du rendu graphique et de la logique de navigation du robot. Cette partie exploite les données générées en C pour animer une simulation interactive et autonome

4.1. Modélisation du Robot

Le robot est conçu comme une entité autonome via la classe **Robot** dans **robot.py**. L'utilisation de la Programmation Orientée Objet (POO) permet une gestion propre et modulaire :

- **Encapsulation** : Toutes les propriétés (position, orientation, couleur) et les comportements (**avancer**, **tourner**, **aller_a...**) sont regroupés au sein d'un même objet.
- **Abstraction des commandes** : La classe transforme des ordres logiques en mouvements graphiques via la bibliothèque **turtle**.
- **Interaction multilingue** : Le robot intègre un dictionnaire de messages alimenté par les fichiers JSON de configuration, lui permettant de "communiquer" ses actions dans la console en plusieurs langues.

4.2. Moteur d'Environnement : Murs, Ouvertures et Obstacles

Le fichier `environnement.py` assure la création du monde virtuel dans lequel le robot évolue.

- **Dynamisme** : Contrairement à un décor statique, les obstacles (balles) sont générés dynamiquement en lisant le fichier `data/informations_balles.txt`. Une conversion est faite pour passer du repère de l'image (pixels) au repère cartésien de la simulation ($x_{turtle} = y_{img} * 2 - 300$).
- **Gestion des zones** : Le système distingue les murs infranchissables des "ouvertures" (portes). Les fonctions `ajouter_ouverture()` et `ajouter_obstacle()` permettent de construire un entrepôt complexe et paramétrable.

4.3. Algorithmes de Navigation et de Recherche de Balles

Le script `simulation.py` fait office de processeur central pour la prise de décision :

- **Sécurité et Évitement** (`detecter_mur()`) : Avant chaque déplacement, une fonction de vérification calcule si la future position du robot entre en collision avec les parois. Une marge de sécurité de 25 unités est appliquée pour éviter tout contact physique virtuel.
- **Navigation par trigonométrie** : La fonction `chercher_balle()` utilise des calculs mathématiques avancés. Elle calcule l'angle de rotation nécessaire via la fonction `atan2(dy, dx)` et la distance précise à parcourir avec l'hypoténuse (`math.hypot`).
- **Fluidité du mouvement** : Les fonctions `avancer_progressif()` et `tourner_progressif()` décomposent les ordres en petits incréments temporels (`time.sleep`), offrant ainsi une animation fluide et visuellement réaliste lors de l'exécution des commandes vocales ou textuelles.

5. Validation et Résultats

Cette section présente les phases de tests effectuées pour valider la robustesse du système et sa capacité à répondre aux ordres de l'utilisateur.

5.1. Scénarios de tests effectués

Pour valider le projet, nous avons défini trois scénarios types simulant des conditions d'utilisation réelles :

- **Test 1 : Pilotage multimodal (Clavier & Voix)**
 - *Objectif* : Vérifier la transmission correcte des ordres depuis le terminal C et l'assistant Python vers le fichier `action.txt`.
 - *Résultat* : Les commandes comme "advance 100" ou "turn left 90" sont exécutées avec une latence quasi nulle, confirmant l'efficacité de la communication par fichiers tampons.
- **Test 2 : Détection et Ralliement de cible**
 - *Objectif* : Commander au robot de trouver une balle spécifique (exemple: "find_ball green").
 - *Résultat* : Le robot calcule correctement son angle via la trigonométrie, pivote, et s'arrête précisément sur la balle détectée par le module de vision.
- **Test 3 : Sécurité périmétrale**
 - *Objectif* : Tenter d'envoyer le robot heurter un mur.
 - *Résultat* : La fonction `detecter_mur()` intercepte l'ordre, stoppe le robot à 25 unités de la paroi et affiche un message d'alerte, garantissant l'intégrité de la simulation.

5.2. Analyse des résultats obtenus

Les tests démontrent une excellente précision dans le traitement d'image et une fluidité de mouvement satisfaisante. L'utilisation du langage C pour l'analyse des fichiers de logs (`_binarisee.txt`, `_caracteristiques.txt`) a permis de valider que chaque objet dans l'entrepôt est identifié avec ses coordonnées exactes avant le lancement de la simulation.

5.3. Limites rencontrées

Malgré les succès, nous avons identifié quelques points d'amélioration :

- **Environnement bruyant** : Dans un cadre très bruyant, la reconnaissance vocale peut parfois mal interpréter les distances et les couleurs des balles, nécessitant alors l'utilisation du mode textuel plus fiable.
- **Complexité des trajectoires** : Actuellement, le robot se déplace en ligne droite vers sa cible. L'ajout d'un algorithme d'évitement d'obstacles dynamiques (pour contourner une balle sur le chemin d'une autre) serait une évolution intéressante.

6. Gestion de Projet et Organisation

Cette section décrit les méthodes et outils utilisés pour coordonner le travail entre les membres du groupe et assurer la cohérence entre les modules C et Python.

6.1. Répartition des tâches au sein du groupe

Afin d'optimiser le développement, nous avons divisé le projet en pôles de compétences, tout en maintenant des points de synchronisation réguliers :

- **Pôle Langage C & Vision** : Responsable de la structure du `main.c`, du traitement d'image (`image.c`, `objet.c`) et de la gestion de la compilation via le `Makefile`.
- **Pôle Python & Simulation** : Responsable de la création de la classe `Robot.py`, du moteur de navigation dans `simulation.py` et de l'environnement graphique, tracé par `environnement.py`.
- **Pôle Intégration & Multilingue** : Chargé de la communication entre les modules via les fichiers `.txt` et de la gestion des configurations JSON.

6.2. Outils de collaboration

Pour pallier la distance et centraliser les ressources, nous avons utilisé un écosystème d'outils complémentaires :

- **GitHub** : Notre outil principal pour le versioning. Il nous a permis de partager le code source, de suivre les modifications et de garantir que tout le monde travaille sur la dernière version des fichiers `include/` ou `config/`.
- **Instagram** : Utilisé comme canal de communication rapide. Ce choix nous a permis une grande réactivité pour les prises de décision immédiates, le partage de captures d'écran de bugs et la coordination des sessions de travail.
- **Trello** : Pour lister les tâches restantes (To-Do list) et suivre l'avancement de chaque module (finalement abandonné, les deux précédents ont suffi).

6.3. Communication et gestion des versions

Le travail collaboratif sur GitHub a présenté des défis techniques enrichissants :

- **Résolution de conflits** : Nous avons dû apprendre à gérer les divergences lors des "merges", notamment lorsque deux membres modifiaient simultanément le fichier `main.c` ou les fichiers de configuration.
- **Normalisation des données** : La communication via `action.txt` a nécessité une discipline stricte sur le format des messages (exemple: toujours utiliser l'anglais pour les commandes techniques comme `advance` afin que le script Python les comprenne).

7. Bilans Individuels

7.1. Bilan de BACHAA Hajar

Dans le cadre de ce projet, ma mission principale a été la conception et l'implémentation de la classe **Robot** en Python (`robot.py`). Ce travail a constitué un véritable défi d'architecture logicielle, car il s'agissait de traduire des concepts abstraits en comportements graphiques concrets.

Mes réalisations techniques :

- **Modélisation Objet** : J'ai structuré le robot en utilisant le paradigme de la Programmation Orientée Objet (POO). Cela a permis d'encapsuler les attributs (position, angle, vitesse) et de créer des méthodes d'action (`avancer`, `tourner`, `aller_a`) réutilisables par le reste de l'équipe.
- **Gestion du multilingue** : J'ai mis en place un système de dictionnaire dynamique au sein de la classe Robot. Ce mécanisme permet au robot de charger des messages depuis les fichiers JSON, rendant la simulation capable d'interagir en français, anglais ou espagnol sans modification du code source.
- **Intégration Graphique** : J'ai assuré la liaison avec la bibliothèque Turtle pour garantir que chaque mouvement calculé par les algorithmes de navigation soit retranscrit de manière fluide et visuelle.

Compétences acquises :

Ce projet m'a permis d'approfondir mes connaissances en Python, notamment sur la gestion des classes et des méthodes en Programmation Orientée Objet. J'ai particulièrement développé des compétences en interopérabilité logicielle : j'ai dû concevoir la logique de mon code Python pour qu'elle s'interface parfaitement avec les flux de données générés par les modules en Langage C. Cette expérience m'a appris à normaliser des structures de données afin d'assurer une communication fluide entre deux environnements de programmation différents. Sur le plan humain, la gestion du versioning via GitHub m'a apporté la rigueur nécessaire au travail collaboratif sur un projet de grande envergure.

Sur le plan collaboratif, j'ai appris à utiliser GitHub, un outil que je ne connaissais pas du tout au départ. Aujourd'hui, je maîtrise les étapes essentielles : travailler sur une branche dédiée, mettre à jour mon code et synchroniser mon travail avec celui de l'équipe sans erreur. Cette expérience m'a appris la rigueur nécessaire pour coder à plusieurs sur un même projet.

7.2. Bilan de BEN LTAIEF Yasmine

Implication dans le déroulement du projet

- Participation active au projet fil rouge réalisé en groupe, en respectant le cahier des charges fourni.
- Travail réalisé en collaboration avec Nidal sur toute la chaîne de traitement des commandes vocales, depuis la reconnaissance vocale jusqu'au traitement final de la commande.
- Préparation de l'environnement de développement et installation des bibliothèques nécessaires au fonctionnement du module de reconnaissance vocale.
- Implémentation et adaptation du code Python fourni pour le module de parole, avec phases de tests et de modifications afin d'obtenir le résultat attendu.
- Résolution d'un problème matériel lié au microphone intégré grâce à l'utilisation de WO Mic pour exploiter le microphone du téléphone.
- Développement de la partie en langage C :
 - réception des commandes vocales depuis un fichier texte
 - filtrage des mots
 - recherche des mots-clés à partir d'un fichier JSON
 - traitement de la commande
- Utilisation de Git pour la gestion des versions, le suivi du projet et le travail collaboratif.

Difficultés rencontrées

- Difficultés initiales liées à la configuration du matériel audio, empêchant l'exécution correcte du module de reconnaissance vocale.
- Complexité de l'intégration entre les parties Python et C, nécessitant des ajustements et des tests répétés.
- Première expérience sur un projet en langage C, impliquant un temps d'apprentissage supplémentaire (syntaxe, gestion des fichiers, logique du programme).
- Problèmes de communication et de coordination, notamment pour lier les travaux des différents membres du groupe.
- Amélioration progressive de la communication, en particulier avec Nidal, permettant de surmonter les difficultés d'intégration et d'assurer la cohérence du projet.

Compétences acquises

- Acquisition de compétences en reconnaissance et traitement des commandes vocales.
- Renforcement des compétences en programmation Python.
- Découverte et apprentissage du langage C à travers un projet concret.

- Manipulation de fichiers texte et JSON et implémentation de la recherche par mots-clés.
- Utilisation de Git pour la gestion de versions dans un projet collaboratif.
- Développement de compétences transversales : travail en équipe, communication, organisation, autonomie et résolution de problèmes.

7.2. Bilan de Devaud Antoine

Travail effectué au cours du projet

Travail initial

Au cours de ce projet, mon rôle a évolué au fur et à mesure de son avancée. Initialement, je devais m'occuper de la **partie configuration**, qui permet le lien entre la **simulation**, la **commande vocale**, la **détection d'image** et la gestion de paramètres changeables et paramétrables, comme des **seuils** de distance ou de couleurs.

Cette tâche a été présente tout au long de l'avancée du projet, car elle nécessitait des modifications régulières du fichier de configuration de base, par exemple pour ajouter des seuils, ou encore des fichiers de **configuration** des langues (fr.json, en.json, es.json), en ajoutant de nouveaux mots-clés avec leurs synonymes possibles afin de les implémenter dans les commandes du robot.

De plus, au départ, avec Thomas, nous avons travaillé sur la réflexion autour de toutes les fonctions indispensables pour la **détection d'image**, allant de la binarisation des pixels jusqu'à la détection de formes d'objets, afin d'être le plus précis possible sur le codage à effectuer.

Travail d'intégration et de coordination

Après certaines de ces fonctions effectuées, j'ai pris le rôle d'**intermédiaire** entre les différents modules afin de les comprendre et de réfléchir aux modifications ou ajouts à réaliser. J'ai donc travaillé avec mes autres camarades sur de nombreux fichiers.

Par exemple, pour la réception des commandes écrites dans le fichier *action.txt*, j'ai **codé le programme simulation**, qui interprète ces mots en les filtrant pour ensuite exécuter, en conséquence, les fonctions du fichier *robot*. Ce fichier a été de nombreuses fois adapté.

En effet, avec Nidal, nous avons travaillé sur le **lien** entre la **commande vocale** et son **interprétation**, avec la création et la modification des fichiers **Python** et **C** liés à la commande vocale et à sa **simulation**, afin que chaque module soit correctement adapté. J'ai réalisé un travail similaire avec Thomas pour lier sa **détection des objets** à l'**affichage** de ces derniers dans la **simulation Turtle**. Cela inclut la création de fonctions permettant l'envoi d'un fichier texte contenant les coordonnées et la

couleur de l'objet, ainsi que des fonctions en **Python** recevant ce fichier jusqu'à l'exécution de l'action correspondante par le robot.

Ajustements finaux et tests

Enfin, à cela s'ajoute la **modification** de certains programmes en **C** et en **Python** pour assurer le bon fonctionnement de la simulation dans chaque module : image, commande vocale et simulation Python.

Finalement, la **phase de test** et de **gestion des bugs** a impliqué toute l'équipe afin de rechercher un maximum de problèmes et de failles, pour les corriger au mieux et obtenir le meilleur résultat possible.

Difficultés rencontrées et compétences acquises

Programmation

Durant ce projet, j'ai dû utiliser **GitHub** pour le partage du code entre coéquipiers et sa mise à jour. Partant de zéro, l'**apprentissage** de cet outil s'est effectué au fur et à mesure de l'avancée globale, avec une compréhension finale non négligeable de son utilisation.

De plus, comme je l'ai expliqué, j'ai dû effectuer le **lien entre les fichiers codés en C et ceux en Python**, ce qui m'a d'abord demandé une compréhension des deux parties et de ce que l'on souhaite exécuter. Je code en Python depuis plusieurs années, mais je ne code en **C** que depuis **cette année**. Cela m'a permis **d'intensifier mon apprentissage** de ce langage dans des conditions réelles, sur un **projet concret**, avec l'aide de mon équipe.

J'ai aussi découvert ce qu'est un **fichier de configuration**, puis appris à les créer. Le principe n'est pas compliqué, mais concevoir des fichiers **JSON** bien optimisés pour chaque module de l'équipe est bien plus complexe et m'a demandé une **compréhension approfondie** de l'ensemble des programmes du **système**.

Gestion d'équipe

Personnellement, venant de classe préparatoire aux grandes écoles, je n'avais jamais réalisé de projet en groupe de cette envergure. J'ai donc **découvert** de nombreux aspects jusque-là inconnus, avec une **équipe** de cinq personnes et un professeur jouant le **rôle du client**.

J'ai ainsi appris de manière globale comment s'**organise** une équipe sur un projet : répartition des rôles, **gestion** des rendez-vous avec le client ou seulement avec l'équipe, ainsi que la **communication interne**.

En effet, tout n'a pas été parfait, avec des **désaccords** sur certains points, mais cela est normal. Ce travail m'a permis de **comprendre que communiquer** au maximum est la meilleure **solution** : plus on communique, plus on évite les **confusions**. Cela

fonctionne également avec le client, et nous l'avons rapidement compris, ce qui nous a amenés à **organiser**, à un moment donné, **un rendez-vous par semaine** afin de nous recentrer sur le projet.

7.4. Bilan de GRELET Thomas

Introduction

Le projet a débuté par la rédaction du Dossier de Spécification, une étape clé à laquelle l'ensemble de l'équipe a activement participé. Une fois ce document validé, j'ai pris en charge la création du dépôt Git, outil central pour le partage et la gestion du code source. Cette étape marque le début d'une phase de travail plus individuelle, dans laquelle chaque membre de l'équipe a pu avancer un peu de son côté tout en s'inscrivant dans une structure commune.

Contribution au projet

1. Organisation et communication

A partir de la phase de code, les prises de rendez-vous et autres échanges avec notre tuteur étaient faits par moi-même, au nom de l'équipe, par le biais de Gmail (convenu avec le tuteur). La communication au sein de l'équipe était faite essentiellement par Instagram, ou en physique.

Cette "partie image" a rapidement eu sa propre branche sur Git, l'occasion d'apprendre comment appréhender ce genre de méthode de développement. Des noms de commits plus appropriés sont également apparus au fur et à mesure du projet.

2. Programmation

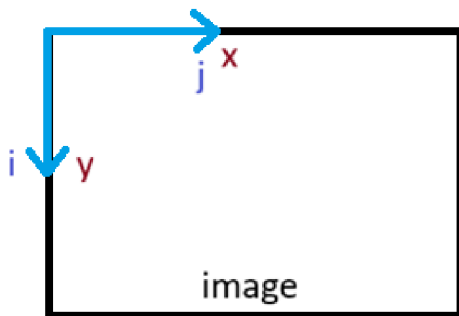
J'ai développé (avec l'aide d'Antoine au début), le module de Traitement d'Image, capable de charger une image pour la binariser (le fond mis à 0 et les objets à 1). Ensuite vient une étape de labellisation (premier objet mis à 1, deuxième objet à 2, etc.). Cela permet de séparer les objets pour les traiter : couleur (grâce à une quantification de chaque pixel, puis un calcul de l'histogramme pour en trouver la couleur dominante), et forme (calcul de la circularité d'un objet pour le classer comme étant une balle ou non). Enfin, des caractéristiques avancées ont été développées concernant les objets, tels que leur centre et leur position sur l'image.

La partie intégration, qui consiste à assembler tous les différents modules ensemble, m'a occupée sur les dernières semaines, où mon rôle a notamment été d'implémenter le traitement de l'image aux commandes (textuelles et vocales) avec la simulation python. Pour cela, Hajar et Antoine m'ont aidé à réaliser des fonctions manquantes, afin d'assurer une bonne complémentarité des différents modules.

Enfin, une fois le tout assemblé, l'ensemble du groupe a pu réfléchir à différents scénarios à présenter le jour de la validation. J'ai donc participé à des séances de tests et de répétitions pour vérifier que tout fonctionne correctement.

Difficultés rencontrées

Au début de la phase de développement, on a pensé à 3 structures de données : une pour l'image, une pour la forme, et enfin une pour l'objet. Mais plus le projet avançait, plus l'utilisation de la structure Forme était floue : la plupart des fonctions concernaient l'image et l'objet mais pas la forme. Finalement, vers la fin de cette phase, j'ai décidé de fusionner la structure Forme avec Objet, supprimant par la même occasion deux fichiers : `forme.h` et `forme.c`.



Une des difficultés a été la façon de représenter les coordonnées d'un pixel sur une image : deux systèmes étaient utilisés en même temps. Le parcours ligne puis colonne (i-j), comme on parcourt naturellement une matrice, et à l'inverse, le x-y, pour les calculs. Cela a mené à des confusions dans l'utilisation des coordonnées lors de calculs. La solution trouvée a été d'utiliser toujours le même système, le i-j a donc été gardé.

L'utilisation du dépôt Git, et particulièrement le système de branche, a été difficile à appréhender. En effet, une fois ma partie bien avancée j'ai voulu merge la branche intitulée "image" dans la branche principale "main", mais des conflits de merge sont apparus. J'ai été un peu perdu sur comment gérer ça au début, mais j'ai surtout appris à y faire face.

Compétences acquises

Durant ces mois de travail, beaucoup de nouvelles notions concernant la Gestion d'équipe et de projet ont été exploitées, comme la communication dans l'équipe, la compréhension d'un cahier des charges, les relations avec un "client" etc.

D'un point de vue technique, en plus d'avoir acquis des connaissances sur le traitement d'image, j'ai pu approfondir mes notions en langage C. J'ai de plus compris comment faire le lien entre plusieurs modules et scripts (C / Python / JSON / txt).

Une expérience en plus avec Git ! Cet outil me semble désormais moins obscur, mais j'ai certainement encore beaucoup à apprendre.

Conclusion personnelle

En conclusion, ce projet a été une expérience enrichissante tant sur le plan technique qu'humain. Il m'a permis de consolider mes compétences en programmation et en traitement d'image, tout en développant des aptitudes essentielles en gestion de projet, en communication et en travail d'équipe. Malgré les difficultés rencontrées, les solutions mises en place ont contribué à la réussite globale du projet et à mon apprentissage personnel.

7.5. Bilan de YAHYAOUUI Nidal

Implication dans le déroulement du projet

Dans ce projet PFR, je me suis principalement occupé du **module de commande vocale et textuelle**, en particulier du fichier **commande_vocale.c**, qui joue un rôle central dans le fonctionnement global du système. Mon travail consistait à faire le lien entre ce que l'utilisateur dit (ou écrit) et ce que le robot est réellement capable d'exécuter.

Concrètement, j'ai développé toute la logique de **filtrage des mots**, d'analyse des phrases et de transformation des commandes en **mots-clés normalisés**. L'objectif était de rendre le système suffisamment robuste pour comprendre différentes formulations, tout en restant compatible avec la simulation Python. Cette partie s'appuie fortement sur des **fichiers JSON**, utilisés pour gérer le vocabulaire, les langues et les unités.

J'ai travaillé principalement avec **Yasmine** sur toute la chaîne de la commande vocale, depuis la récupération de la phrase issue de la reconnaissance vocale en Python jusqu'au traitement final en C. Nous avons souvent travaillé ensemble pour tester, corriger et ajuster nos deux parties afin qu'elles communiquent correctement. À plusieurs reprises, nous avons également échangé avec notre tuteur **Julien VANDERSTRAETEN** lors de visios, notamment lorsque certaines décisions techniques posaient problème ou lorsque le comportement du programme n'était pas celui attendu.

Difficultés rencontrées

Honnêtement, cette partie du projet a été **l'une des plus compliquées**. Il y a eu de nombreuses situations où **le programme ne fonctionnait pas du tout**, ou fonctionnait partiellement sans raison évidente. Il arrivait par exemple que la commande vocale soit bien

reconnue côté Python, mais totalement mal interprétée en C, ou qu'une commande simple fonctionne alors qu'une commande légèrement différente provoque une erreur ou un comportement incohérent.

Le **filtrage de mots en C**, combiné à la lecture de fichiers JSON, a été particulièrement difficile. La gestion des chaînes de caractères, de la mémoire, des commandes composées (*avance et tourne, go to x y, etc.*) et des valeurs implicites (distance ou angle par défaut) a demandé énormément de tests et de débogage. Plusieurs fois, après des heures de travail, le programme ne marchait toujours pas, ce qui obligeait à reprendre le raisonnement depuis le début.

Sur le plan organisationnel, les **dernières semaines ont été intenses**. L'intégration de tous les modules en même temps a révélé des problèmes de communication : formats de fichiers mal compris, attentes différentes entre les modules C et Python, conflits Git ou modifications écrasées. Il a parfois fallu se débrouiller rapidement, corriger en urgence et adapter son code à celui des autres pour que l'ensemble fonctionne enfin correctement.

Compétences acquises

Malgré ces difficultés, ce projet m'a permis de progresser énormément. J'ai acquis une bien meilleure maîtrise du **langage C**, notamment sur la manipulation de chaînes, la structuration d'un code complexe et le débogage. J'ai aussi appris à travailler avec des **fichiers JSON** dans un contexte réel et à assurer une **communication fiable entre deux langages différents (C et Python)**.

Sur le plan humain, ce projet m'a appris à **persévérer quand rien ne fonctionne**, à mieux communiquer avec les autres membres du groupe et à m'adapter aux contraintes collectives. Cette expérience se rapproche fortement d'un contexte professionnel, où tout ne fonctionne pas du premier coup et où il faut souvent trouver des solutions rapidement.

8. Conclusion

8.1. Bilan général du projet

Le Projet Fil Rouge a été une expérience technique complète, nous permettant de franchir les étapes, allant de la conception théorique à la réalisation d'une simulation fonctionnelle. Nous avons réussi à lever le défi majeur de l'interopérabilité entre le Langage C (performant pour le traitement de données et d'images) et Python (flexible pour la logique de contrôle et le rendu graphique).

Le système final remplit l'ensemble des objectifs initiaux : le robot est capable de recevoir des ordres via une interface multimodale (voix et texte), d'analyser son environnement grâce à un module de vision dédié, et de naviguer de manière autonome et sécurisée dans un entrepôt virtuel.

8.2. Perspectives d'évolution

Bien que le système soit opérationnel, plusieurs pistes pourraient être explorées pour aller plus loin :

- **Intelligence Artificielle** : Remplacer la détection par binarisation de couleur par un modèle de Deep Learning pour identifier des objets plus complexes.
- **Navigation Avancée** : Implémenter un algorithme de type A* (A-star) pour permettre au robot de calculer le chemin le plus court tout en contournant des obstacles mobiles.
- **Passage au Réel** : Transposer cette logique logicielle sur un robot physique (type Raspberry Pi avec caméra) pour tester la robustesse des algorithmes face aux contraintes du monde réel (luminosité variable, frottements au sol).